LEZIONE 15 - Funzioni: Parte 1

PARTE TEORICA (2h)

1. Concetto di Funzione e Modularità

Una **funzione** è un blocco di codice riutilizzabile che esegue un compito specifico. Le funzioni sono il fondamento della **programmazione modulare**.

Vantaggi delle funzioni: - **Riutilizzabilità**: scrivi una volta, usa molte volte - **Modularità**: divide programmi complessi in parti gestibili - **Manutenibilità**: più facile trovare e correggere errori - **Leggibilità**: codice più chiaro e organizzato - **Testing**: puoi testare singole funzioni - **Collaborazione**: diversi programmatori su funzioni diverse

Analogia della vita reale:

```
Preparare una pizza:
├─ Preparare l'impasto()
├─ Stendere la pasta()
├─ Aggiungere ingredienti()
└─ Cuocere in forno()

Ogni fase è una "funzione" con un compito specifico!
```

Esempio senza funzioni (codice duplicato):

```c
// Calcolo area rettangolo 1
int base1 = 5, altezza1 = 3;
int area1 = base1 * altezza1;
printf("Area 1: %d\n", area1);

// Calcolo area rettangolo 2
int base2 = 8, altezza2 = 4;
int area2 = base2 * altezza2;
printf("Area 2: %d\n", area2);

// Calcolo area rettangolo 3
int base3 = 10, altezza3 = 6;
int area3 = base3 * altezza3;
printf("Area 3: %d\n", area3);
```

Esempio con funzioni (codice riutilizzabile):

```c
int calcolaArea(int base, int altezza) {
    return base * altezza;
}

int main() {
    printf("Area 1: %d\n", calcolaArea(5, 3));
    printf("Area 2: %d\n", calcolaArea(8, 4));
    printf("Area 3: %d\n", calcolaArea(10, 6));
    return 0;
}
```

---

2. Definizione e Dichiarazione

Struttura di una funzione:

```
tipoRitorno nomeFunzione(parametro1, parametro2, ...) {
    // corpo della funzione
    // istruzioni
    return valore;  // se tipoRitorno != void
}
```

Componenti: 1. **Tipo di ritorno**: tipo del valore restituito (int, float, void, ecc.) 2. **Nome della funzione**: identificatore univoco (convenzioni camelCase o snake_case) 3. **Parametri**: input della funzione (opzionali) 4. **Corpo**: blocco di codice tra {} 5. **Return**: restituisce un valore (obbligatorio se tipo != void)

Esempi:

```
// Funzione che calcola la somma di due numeri
int somma(int a, int b) {
    int risultato = a + b;
    return risultato;
}

// Funzione che stampa un saluto (nessun ritorno)
void saluta(char nome[]) {
    printf("Ciao, %s!\n", nome);
    // Non serve return
}

// Funzione senza parametri
int getNumeroFisso() {
    return 42;
}

// Funzione che verifica se un numero è pari
int isPari(int n) {
    return (n % 2 == 0);  // Ritorna 1 (vero) o 0 (falso)
}
```

Dichiarazione vs Definizione:

```
// DICHIARAZIONE (prototipo): dice al compilatore che la funzione esiste
int somma(int a, int b);

// DEFINIZIONE: implementa la funzione
int somma(int a, int b) {
    return a + b;
}

// Esempio completo
#include <stdio.h>

// Dichiarazioni (in alto)
int somma(int a, int b);
int prodotto(int a, int b);

int main() {
    printf("%d\n", somma(5, 3));      // 8
    printf("%d\n", prodotto(5, 3));   // 15
    return 0;
}

// Definizioni (dopo main)
int somma(int a, int b) {
    return a + b;
}

int prodotto(int a, int b) {
    return a * b;
}
```

---

3. Parametri Formali e Attuali

Parametri formali: variabili nella definizione della funzione **Parametri attuali (o argomenti)**: valori passati quando si chiama la funzione

```c
// a e b sono PARAMETRI FORMALI
int somma(int a, int b) {
    return a + b;
}

int main() {
    int x = 10, y = 20;

    // x e y sono PARAMETRI ATTUALI (argomenti)
    int risultato = somma(x, y);

    // 5 e 3 sono PARAMETRI ATTUALI (valori letterali)
    int risultato2 = somma(5, 3);

    return 0;
}
```

Corrispondenza parametri:

```c
int calcola(int primo, int secondo, int terzo) {
    return primo + secondo * terzo;
}

int main() {
    int risultato = calcola(10, 5, 2);
    // primo = 10
    // secondo = 5
    // terzo = 2
    // ritorna: 10 + 5 * 2 = 20

    return 0;
}
```

ATTENZIONE all'ordine:

```c
int sottrai(int a, int b) {
    return a - b;
}

int main() {
    printf("%d\n", sottrai(10, 3));  // 7
    printf("%d\n", sottrai(3, 10));  // -7 (DIVERSO!)
    return 0;
}
```

---

4. Return: Restituzione del Valore

Il comando **return** termina l'esecuzione della funzione e restituisce un valore al chiamante.

Sintassi:

```c
return espressione;
```

Esempi:

```c
// Return con valore semplice
int getNumero() {
    return 42;
}

// Return con espressione
int quadrato(int n) {
    return n * n;
}

// Return con calcolo
int massimo(int a, int b) {
    if (a > b) {
        return a;
    } else {
        return b;
    }
}

// Return multipli (la funzione esce al primo return)
int segno(int n) {
    if (n > 0) return 1;
    if (n < 0) return -1;
    return 0;
}

// Return anticipato
int divide(int a, int b) {
    if (b == 0) {
        printf("Errore: divisione per zero!\n");
        return 0;  // Esce subito
    }
    return a / b;
}
```

Funzioni void (senza return):

```c
void stampaQuadrato(int n) {
    printf("%d\n", n * n);
    // Non serve return
}

void stampaMessaggio() {
    printf("Ciao!\n");
    return;  // Opzionale, esce dalla funzione
}

void esempio(int x) {
    if (x < 0) {
        printf("Numero negativo!\n");
        return;  // Esce subito senza fare altro
    }
    printf("Numero valido: %d\n", x);
}
```

ERRORI COMUNI:

```c
// ERRORE: funzione int senza return
int sbagliata() {
    int x = 10;
    // Manca return! Comportamento indefinito
}

// ERRORE: tipo di ritorno sbagliato
int altraFunzione() {
    return 3.14;  // Ritorna int, ma 3.14 è float (viene troncato a 3)
}

// CORRETTO: conversione esplicita
float funzioneFloat() {
    return 3.14;  // OK, tipo corretto
}
```

---

5. Passaggio per Valore

In C, i parametri vengono passati **per valore**: la funzione riceve una **copia** dei dati, non gli originali.

Conseguenza: modificare un parametro NON modifica la variabile originale.

```c
void incrementa(int x) {
    x = x + 1;
    printf("Dentro la funzione: x = %d\n", x);
}

int main() {
    int numero = 10;

    incrementa(numero);
    // Output: Dentro la funzione: x = 11

    printf("Fuori dalla funzione: numero = %d\n", numero);
    // Output: Fuori dalla funzione: numero = 10
    // numero NON è cambiato!

    return 0;
}
```

Spiegazione visiva:

```
main():                    incrementa():
┌──────────┐               ┌──────────┐
│ numero=10│ ----copia--→  │   x=10   │
└──────────┘               └──────────┘
                                ↓
                           x diventa 11
                                ↓
                           funzione termina
                           x viene distrutto

numero rimane 10!
```

Come restituire il valore modificato:

```c
// Soluzione: usare return
int incrementa(int x) {
    return x + 1;
}

int main() {
    int numero = 10;
    numero = incrementa(numero);  // Assegna il nuovo valore
    printf("numero = %d\n", numero);  // Output: 11
    return 0;
}
```

Passaggio di array (caso speciale):

```c
void modificaArray(int arr[], int dim) {
    arr[0] = 999;  // Questo MODIFICA l'array originale!
}

int main() {
    int numeri[] = {1, 2, 3};
    modificaArray(numeri, 3);
    printf("%d\n", numeri[0]);  // Output: 999
    // Gli array vengono passati per riferimento (vedremo meglio dopo)
    return 0;
}
```

---

6. Scope delle Variabili

Lo **scope** (ambito) di una variabile definisce dove può essere usata.

Tipi di scope:

1. Variabili locali (dentro funzioni):

```c
void funzione1() {
    int x = 10;  // x è locale a funzione1
    printf("%d\n", x);  // OK
}

void funzione2() {
    printf("%d\n", x);  // ERRORE! x non esiste qui
}

int main() {
    printf("%d\n", x);  // ERRORE! x non esiste qui
    funzione1();
    return 0;
}
```

2. Variabili globali (fuori da tutte le funzioni):

```c
#include <stdio.h>

int globale = 100;  // Variabile globale

void stampa() {
    printf("globale = %d\n", globale);  // OK
}

void modifica() {
    globale = 200;  // OK, modifica la variabile globale
}

int main() {
    printf("globale = %d\n", globale);  // 100
    modifica();
    printf("globale = %d\n", globale);  // 200
    stampa();                           // 200
    return 0;
}
```

3. Parametri (locali alla funzione):

```c
void funzione(int param) {
    param = 100;  // param è locale
    printf("%d\n", param);
}

int main() {
    funzione(50);
    // printf("%d\n", param);  // ERRORE! param non esiste qui
    return 0;
}
```

Shadowing (ombreggiamento):

```c
int x = 100;  // Globale

void funzione() {
    int x = 50;  // Locale, "nasconde" la globale
    printf("x locale = %d\n", x);  // 50
}

int main() {
    printf("x globale = %d\n", x);  // 100
    funzione();                     // 50
    printf("x globale = %d\n", x);  // 100 (globale non modificata)
    return 0;
}
```

Best practices: - Preferisci variabili locali - Usa variabili globali solo se necessario - Evita nomi ambigui che causano shadowing - Ogni funzione dovrebbe avere responsabilità chiare

---

PARTE PRATICA (2h)

Esercizio 1: Funzioni Matematiche Base

```c
#include <stdio.h>

// Dichiarazioni
int somma(int a, int b);
int sottrazione(int a, int b);
int moltiplicazione(int a, int b);
float divisione(int a, int b);
int potenza(int base, int esponente);
int fattoriale(int n);

int main() {
    int x = 10, y = 5;

    printf("=== CALCOLATRICE ===\n");
    printf("%d + %d = %d\n", x, y, somma(x, y));
    printf("%d - %d = %d\n", x, y, sottrazione(x, y));
    printf("%d * %d = %d\n", x, y, moltiplicazione(x, y));
    printf("%d / %d = %.2f\n", x, y, divisione(x, y));
    printf("%d ^ %d = %d\n", x, y, potenza(x, y));
    printf("%d! = %d\n", y, fattoriale(y));

    return 0;
}

// Definizioni
int somma(int a, int b) {
    return a + b;
}

int sottrazione(int a, int b) {
    return a - b;
}

int moltiplicazione(int a, int b) {
    return a * b;
}

float divisione(int a, int b) {
    if (b == 0) {
        printf("Errore: divisione per zero!\n");
        return 0.0;
    }
    return (float)a / b;
}

int potenza(int base, int esponente) {
    int risultato = 1;
    for (int i = 0; i < esponente; i++) {
        risultato *= base;
    }
    return risultato;
}

int fattoriale(int n) {
    if (n < 0) return -1;  // Errore
    if (n == 0 || n == 1) return 1;

    int risultato = 1;
    for (int i = 2; i <= n; i++) {
        risultato *= i;
    }
    return risultato;
}
```

Esercizio 2: Funzioni per Validazione

```c
#include <stdio.h>

// Dichiarazioni
int isPari(int n);
int isDispari(int n);
int isPrimo(int n);
int isPerfetto(int n);
int isPositivo(int n);
int isNegativo(int n);

int main() {
    int numeri[] = {2, 7, 15, 28, -5, 0, 11, 6};
    int dim = 8;

    printf("=== VALIDAZIONE NUMERI ===\n\n");

    for (int i = 0; i < dim; i++) {
        int n = numeri[i];
        printf("Numero: %d\n", n);
        printf("  Pari: %s\n", isPari(n) ? "SI" : "NO");
        printf("  Dispari: %s\n", isDispari(n) ? "SI" : "NO");
        printf("  Primo: %s\n", isPrimo(n) ? "SI" : "NO");
        printf("  Perfetto: %s\n", isPerfetto(n) ? "SI" : "NO");
        printf("  Positivo: %s\n", isPositivo(n) ? "SI" : "NO");
        printf("\n");
    }

    return 0;
}

// Definizioni
int isPari(int n) {
    return n % 2 == 0;
}

int isDispari(int n) {
    return n % 2 != 0;
}

int isPrimo(int n) {
    if (n < 2) return 0;
    if (n == 2) return 1;
    if (n % 2 == 0) return 0;

    for (int i = 3; i * i <= n; i += 2) {
        if (n % i == 0) return 0;
    }
    return 1;
}

int isPerfetto(int n) {
    if (n <= 0) return 0;

    int somma = 0;
    for (int i = 1; i < n; i++) {
        if (n % i == 0) {
            somma += i;
        }
    }
    return somma == n;
}

int isPositivo(int n) {
    return n > 0;
}

int isNegativo(int n) {
    return n < 0;
}
```

Esercizio 3: Funzioni per Array

```c
#include <stdio.h>

#define MAX 100

// Dichiarazioni
void stampaArray(int arr[], int dim);
int sommaArray(int arr[], int dim);
float mediaArray(int arr[], int dim);
int massimoArray(int arr[], int dim);
int minimoArray(int arr[], int dim);
int cercaElemento(int arr[], int dim, int target);
int contaPari(int arr[], int dim);
void invertiArray(int arr[], int dim);

int main() {
    int numeri[] = {15, 8, 23, 42, 7, 19, 31, 5, 12, 28};
    int dim = 10;

    printf("Array originale:\n");
    stampaArray(numeri, dim);

    printf("\nSomma: %d\n", sommaArray(numeri, dim));
    printf("Media: %.2f\n", mediaArray(numeri, dim));
    printf("Massimo: %d\n", massimoArray(numeri, dim));
    printf("Minimo: %d\n", minimoArray(numeri, dim));
    printf("Numeri pari: %d\n", contaPari(numeri, dim));

    int target = 19;
    int pos = cercaElemento(numeri, dim, target);
    if (pos != -1) {
        printf("Elemento %d trovato in posizione %d\n", target, pos);
    } else {
        printf("Elemento %d non trovato\n", target);
    }

    printf("\nArray invertito:\n");
    invertiArray(numeri, dim);
    stampaArray(numeri, dim);

    return 0;
}

// Definizioni
void stampaArray(int arr[], int dim) {
    printf("[ ");
    for (int i = 0; i < dim; i++) {
        printf("%d ", arr[i]);
    }
    printf("]\n");
}

int sommaArray(int arr[], int dim) {
    int somma = 0;
    for (int i = 0; i < dim; i++) {
        somma += arr[i];
    }
    return somma;
}

float mediaArray(int arr[], int dim) {
    if (dim == 0) return 0.0;
    return (float)sommaArray(arr, dim) / dim;
}

int massimoArray(int arr[], int dim) {
    if (dim == 0) return 0;
```

```c
    int max = arr[0];
    for (int i = 1; i < dim; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}

int minimoArray(int arr[], int dim) {
    if (dim == 0) return 0;

    int min = arr[0];
    for (int i = 1; i < dim; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
    }
    return min;
}

int cercaElemento(int arr[], int dim, int target) {
    for (int i = 0; i < dim; i++) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;  // Non trovato
}

int contaPari(int arr[], int dim) {
    int contatore = 0;
    for (int i = 0; i < dim; i++) {
        if (arr[i] % 2 == 0) {
            contatore++;
        }
    }
    return contatore;
}

void invertiArray(int arr[], int dim) {
    for (int i = 0; i < dim / 2; i++) {
        int temp = arr[i];
        arr[i] = arr[dim - 1 - i];
        arr[dim - 1 - i] = temp;
    }
}
```

Esercizio 4: Libreria di Utility

```c
#include <stdio.h>
#include <math.h>

// Dichiarazioni - Conversioni
float celsiusToFahrenheit(float celsius);
float fahrenheitToCelsius(float fahrenheit);
float kmToMiles(float km);
float milesToKm(float miles);

// Dichiarazioni - Geometria
float areaRettangolo(float base, float altezza);
float areaTriangolo(float base, float altezza);
float areaCerchio(float raggio);
float perimetroRettangolo(float base, float altezza);
float perimetroQuadrato(float lato);
float circonferenza(float raggio);
```

```c
// Dichiarazioni - Matematica
int massimoDi3(int a, int b, int c);
int minimoDi3(int a, int b, int c);
float distanza2D(float x1, float y1, float x2, float y2);

int main() {
    // Test conversioni
    printf("=== CONVERSIONI ===\n");
    printf("25°C = %.2f°F\n", celsiusToFahrenheit(25));
    printf("77°F = %.2f°C\n", fahrenheitToCelsius(77));
    printf("10 km = %.2f miglia\n", kmToMiles(10));
    printf("5 miglia = %.2f km\n", milesToKm(5));

    // Test geometria
    printf("\n=== GEOMETRIA ===\n");
    printf("Area rettangolo (5x3): %.2f\n", areaRettangolo(5, 3));
    printf("Area triangolo (6x4): %.2f\n", areaTriangolo(6, 4));
    printf("Area cerchio (r=5): %.2f\n", areaCirc chio(5));
    printf("Perimetro rettangolo (5x3): %.2f\n", perimetroRettangolo(5, 3)

    // Test matematica
    printf("\n=== MATEMATICA ===\n");
    printf("Massimo tra 10, 25, 18: %d\n", massimoDi3(10, 25, 18));
    printf("Minimo tra 10, 25, 18: %d\n", minimoDi3(10, 25, 18));
    printf("Distanza tra (0,0) e (3,4): %.2f\n", distanza2D(0, 0, 3, 4));

    return 0;
}

// Definizioni - Conversioni
float celsiusToFahrenheit(float celsius) {
    return celsius * 9.0 / 5.0 + 32.0;
}

float fahrenheitToCelsius(float fahrenheit) {
    return (fahrenheit - 32.0) * 5.0 / 9.0;
}

float kmToMiles(float km) {
    return km * 0.621371;
}

float milesToKm(float miles) {
    return miles / 0.621371;
}

// Definizioni - Geometria
float areaRettangolo(float base, float altezza) {
    return base * altezza;
}

float areaTriangolo(float base, float altezza) {
    return (base * altezza) / 2.0;
}

float areaCerchio(float raggio) {
    return 3.14159 * raggio * raggio;
}

float perimetroRettangolo(float base, float altezza) {
    return 2 * (base + altezza);
}

float perimetroQuadrato(float lato) {
    return 4 * lato;
}

float circonferenza(float raggio) {
    return 2 * 3.14159 * raggio;
}
```

```c
// Definizioni - Matematica
int massimoDi3(int a, int b, int c) {
    int max = a;
    if (b > max) max = b;
    if (c > max) max = c;
    return max;
}

int minimoDi3(int a, int b, int c) {
    int min = a;
    if (b < min) min = b;
    if (c < min) min = c;
    return min;
}

float distanza2D(float x1, float y1, float x2, float y2) {
    float dx = x2 - x1;
    float dy = y2 - y1;
    return sqrt(dx * dx + dy * dy);
}
```

Esercizi da svolgere autonomamente:

1. **Calcolatrice avanzata**: Crea funzioni per calcolare MCD, mcm, radice quadrata (approssimata)
2. **Funzioni stringhe**: Implementa le tue versioni di strlen, strcpy, strcmp senza usare string.h
3. **Statistiche avanzate**: Crea funzioni per calcolare varianza, deviazione standard, mediana
4. **Validazione password**: Funzione che verifica se una password è valida (lunghezza, maiuscole, numeri, caratteri speciali)
5. **Conversione numeri**: Funzioni per convertire decimale→binario e binario→decimale
6. **Giochi**: Crea funzioni per un gioco (es: lancio dadi, morra cinese)
7. **Date**: Funzioni per validare date, calcolare giorni tra due date, verificare anno bisestile
8. **Ordinamento**: Implementa bubble sort, selection sort in funzioni separate
9. **Matrici**: Funzioni per moltiplicare matrici, calcolare determinante, verificare simmetria
10. **Menu interattivo**: Crea un menu che chiama diverse funzioni in base alla scelta dell'utente

---

Riepilogo Concetti Chiave

✓ Le **funzioni** rendono il codice modulare e riutilizzabile
✓ Struttura: `tipo nome(parametri) { corpo; return valore; }`
✓ **Dichiarazione** (prototipo) vs **Definizione** (implementazione)
✓ **Parametri formali** nella definizione, **parametri attuali** nella chiamata
✓ **Passaggio per valore**: la funzione riceve una copia
✓ **Scope**: variabili locali vs globali

Prossima lezione: Approfondiremo le **funzioni ricorsive**, prototipazione, variabili static e best practices!